P O R T F O L I O

박인호 | Game Programmer

NPC 미션 시스템 + 쿼터뷰 디펜스 미니게임 예제

Unity 6 (6000.3.10f1) Environment | **Factory Method · State Machine · Observer · Object Pool · Template Method** Applied

AI Assisted | **Claude Code 활용 개발**

Portfolio/Unity

AI-ASSISTED DEVELOPMENT

AI에게 코드를 맡기는 것이 아니라, 규칙으로 AI를 관리합니다.

☰ 규칙을 파일로 정의해 매번 같은 기준으로 작업

같은 요구사항을 반복해서 설명하지 않습니다. 공통 Skills 파일은 `CLAUDE.md` import로 세션 시작 시 자동 로드되고, 포트폴리오처럼 특정 작업용 규칙은 그 작업 때만 읽어 토큰을 아낍니다.

</> 나만의 코딩 스타일 51개 규칙 + 언어별 표준

C++는 Unreal Engine Coding Standard, C#은 Microsoft 규칙을 따르고 그 위에 내 스타일을 적용 — 어떤 언어든 AI가 쓴 코드가 내 코드처럼 읽힙니다.

↻ 작업할수록 규칙이 쌓이는 구조

반복되는 지적은 AI가 Skills 파일에 규칙으로 추가하고 항상 보고합니다. 진입점 `CLAUDE.md`는 승인 없이는 수정하지 않습니다.

```
CLAUDE.md                    ← 세션 시작 시 자동 로드
└ ClaudeSkills/
   ├ DefaultSkills.md        ← 작업 · 빌드 · 검증 규칙
   ├ CodingSkills.md         ← 코딩 스타일 51개
   ├ TokenSavingSkills.md    ← 토큰 절약 규칙
   └ PortfolioSkills.md      ← 문서 작업 때만 읽음
```

"규칙은 한 번 정하고, AI는 매번 지킵니다."

종류별로 나눈 Skills 파일은 다른 프로젝트에서도 골라 재사용

SAFE AI REFACTORING

기존 프로젝트의 코드를 해치지 않도록, 안전장치를 규칙으로 걸었습니다.

✎

주석과 의도 보존

AI는 주석을 넣거나 고치지 않습니다. 이름 변경으로 주석이 틀려지는 경우는 사람에게 확인한 뒤에만 함께 수정합니다.

</>

C# 적용 기준 명시

Microsoft C# 규칙 위에 내 스타일을 얹되, `var` 금지 · `_camelCase` 필드 · `?.`는 델리게이트 호출에만 등 C#에서 갈리는 부분을 따로 정해 해석 차이를 없앴습니다.

⟳

결과 5회 재검토

작업을 마치면 요청대로 됐는지, 빠진 것 · 틀린 것, 다른 곳에 미치는 영향, 더 나은 방법을 다시 검토한 뒤 결론을 가지고 보고합니다.

⇄

엔진이 달라도 같은 규칙

Unreal과 Unity 프로젝트를 한 저장소에서 같은 Skills 파일로 관리합니다. 엔진 · 언어가 바뀌어도 AI는 같은 기준으로 작업합니다.

?

판단은 사람이

규칙끼리 충돌하면 AI가 임의로 처리하지 않고 확인을 요청합니다. 커밋 · 푸시도 요청할 때만 합니다.

≡

diff 최소화

파일의 인코딩과 줄바꿈을 그대로 유지해, 리뷰할 때 실제 변경만 보이게 합니다.

결과 | Unreal(C++)과 같은 규칙 파일을 그대로 사용 → C#은 언어별 적용 기준만 추가 → 두 엔진의 코드가 같은 스타일로 유지

THE PROGRAMMER'S ROLE WITH AI

프로그래머는 단순히 코드를 작성하는 사람이 아니라, AI와 협업해 개발 생산성을 끌어올리는 사람이어야 합니다.

01 **기준 설계** 규칙 · 안전장치 정의 → 02 **AI 실행** 구현 · 리팩터링 일괄 적용 → 03 **자동 검증** 빌드 · 런타임 테스트 → 04 **리뷰 · 결정** 사람이 최종 판단 → 05 **규칙 축적** 다음 작업에 자동 적용

코드만 작성하는 프로그래머

- 모든 파일을 직접 열어 한 줄씩 수정
- 스타일 통일과 반복 검증에 시간 대부분을 소모
- 작업량이 곧 손의 속도에 묶임

AI를 관리하는 프로그래머

- 기준과 안전장치를 설계하고, 반복 작업은 AI에게 위임
- 절약한 시간을 설계 · 리뷰 · 품질 판단에 집중
- 규칙이 쌓일수록 같은 시간에 내는 성과가 커짐

"코딩은 AI와 함께, 기준과 판단은 프로그래머가."

생산성은 도구가 아니라 도구를 관리하는 방식에서 나옵니다.

MISSION SYSTEM DESIGN GOALS

UI와 도메인 로직의 완전한 분리
핵심 연산과 연출의 독립으로 유지보수 비용 절감

객체 간 직접 참조 최소화
의존성을 제거하여 예기치 못한 사이드 이펙트 방지

상태 기반(State-driven) 설계
미션 라이프사이클의 명확한 관리와 흐름 제어

런타임 비용 최소화
오브젝트 풀링으로 Instantiate/Destroy 비용과 GC 부담 절감

확장성 및 유지보수성 확보
새로운 미션 타입 추가가 용이한 유연한 구조

> **"Simple, Decoupled, Stable"**
>
> *가독성 높은 코드는 비즈니스 로직의 명확성에서 시작됩니다.*

GAMEPLAY LOOP

미션 수락은 곧바로 별도의 쿼터뷰 디펜스 미니게임으로 이어집니다.

01
NPC 접촉
→
02
미션 수락 (Agree)
→
03
디펜스 미니게임
(몬스터 웨이브 생존)
→
04
결과 판정
→
05
결과창 & 원위치 복귀

생존 성공 (Succeeded)

생존 타이머 만료 → `OnSurvived` 브로드캐스트 → 미션이 `SuccessMission()` 호출

HP 소진 (Failed)

몬스터 피격으로 HP 0 도달 → `HandlePlayerDefeated()` → 미션이 `FailedMission()` 호출

ARCHITECTURE & COMMUNICATION FLOW

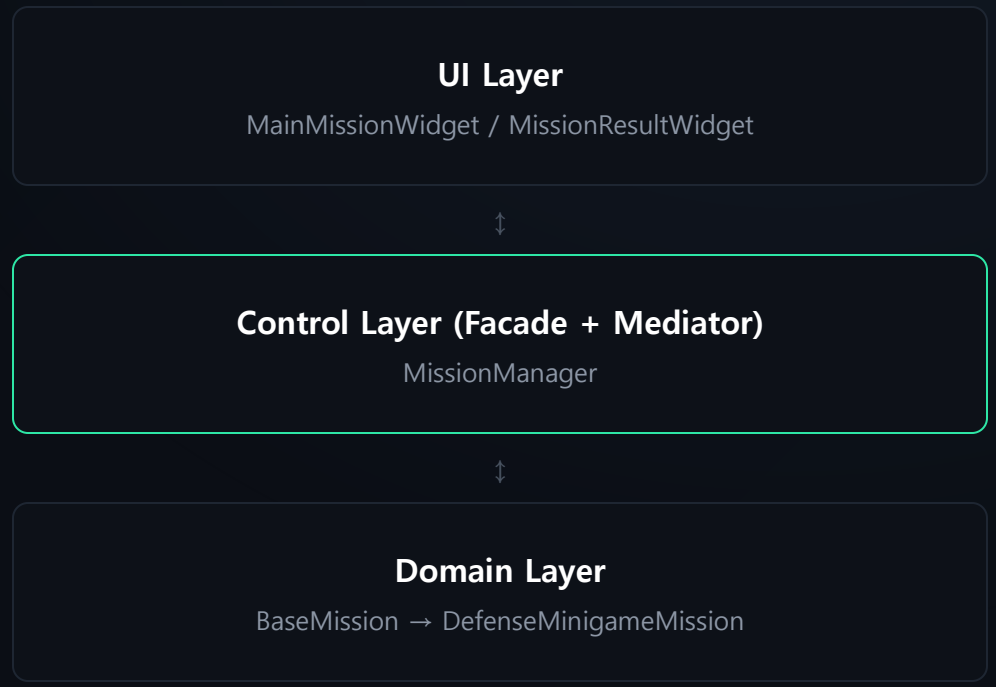
3-Layer + Facade/Mediator Structure

Widget과 Mission은 서로를 전혀 모르는 상태로 유지되며, **MissionManager**가 모든 통신을 중재합니다.

명령 흐름 (Command): 상위(Widget) → 중재자(MissionManager) → 하위(BaseMission)로의 함수 호출

이벤트 흐름 (Event): 하위(BaseMission) 상태 변경 → C# event 브로드캐스트 → 상위(Widget) 갱신

> 디펜스 미니게임(몬스터 스폰 · 생존 타이머)은 MissionManager가 아니라, 미니게임을 사용하는 미션(`DefenseMinigameMission`) 자신이 소유합니다 — 단일 책임 원칙(SRP)에 따라 "어떤 미션이 미니게임을 쓰는가"는 미션 스스로 결정합니다.



CORE DESIGN PATTERNS

Facade / Mediator Pattern

MissionManager가 모든 시스템 간 통신의 유일한 진입점 역할을 하며, 객체 간 직접 참조를 제거합니다.

Observer Pattern

C# event로 상태 변경을 전파하는 Event-driven 방식으로, 매 프레임 폴링 비용을 줄입니다.

Factory Method Pattern

MissionFactory가 MissionUnique를 각 미션 클래스로 매핑, 새 미션 추가 시 분기만 추가하면 됩니다.

State Machine

Ready→InProgress→Succeeded/Failed 전이를 SetState() 한 곳에서 관리해 흐름을 예측 가능하게 만듭니다.

Object Pool Pattern

몬스터를 Destroy 대신 Deactivate로 반환·재사용해, 반복 스폰의 생성 비용과 GC 부담을 줄입니다.

Template Method Pattern

DefenseMinigameMission이 공통 흐름을 정의하고, 각 미션 서브클래스는 Enter*State() 훅만 오버라이드합니다.

MISSION STATE & TERMINATION RULES

미션 상태 (STATE)	트리거	결과
Ready (대기)	Agree 버튼 클릭	`AgreeMission()` → InProgress 진입, 디펜스 미니게임 시작
Ready (대기)	Disagree 버튼 클릭	`DisagreeMission()` → 미션 취소
InProgress (진행 중)	생존 타이머 만료	`SuccessMission()` → Succeeded
InProgress (진행 중)	HP 0 도달	`FailedMission()` → Failed

진행 중(InProgress)에는 버튼으로 미션을 강제 종료할 수 없습니다 — 오직 디펜스 미니게임의 결과만으로 미션이 종료됩니다. 과거 존재했던 수동 Success/Fail 버튼은 이 원칙에 따라 제거되었습니다.

AUTOMATED TEST CYCLE

사람이 직접 플레이하지 않아도, 미션 전체 흐름을 자동으로 검증하는 도구입니다.

F8 키 한 번으로 전체 사이클 실행

NPC 접근 → 수락 → 디펜스 미니게임 → 결과 확인까지 자동 진행

기존 코드는 자동 모드의 존재를 전혀 모름

`MissionManager`가 노출하는 public 함수만 호출 — 실제 게임 로직과 완전히 분리된 구조

Coroutine 기반 프레임 단위 이동

yield return null로 매 프레임 갱신되어, 타이머 기반 이동에서 흔한 프레임 드랍 문제 자체가 생기지 않는 구조

반복 수동 테스트 제거

전체 플로우 회귀 확인에 드는 시간 단축

```
> F8 → ToggleAutoCycleMode()
├ Ready  → HandleMissionButtonAction(Left)
├ InProgress (자동 회피 이동)
├ Succeeded → HandleMissionResultConfirmed()
└ 한 사이클 종료 → 자동 OFF
```

> **"직접 만든 시스템은, 직접 검증할 수 있어야 합니다."**
>
> MissionAutoCycleController — Manager/

THANK YOU

안정적이고 확장 가능한 코드, 팀의 자산이 되는 개발자가 되겠습니다.
AI와 협업해 개발 생산성을 높이는 개발자가 되겠습니다.

hellop1013@naver.com

https://github.com/inhopark/Portfolio